Machine Learning: Naïve Bayes and K-Nearest Neighbor

Dr. Oybek Eraliev,

Department of Computer Engineering

Inha University In Tashkent.

Email: oybekeraliev7@gmail.com

Machine Learning Algorithms

Content

- What is Naïve Bayes?
- Naïve Bayes and Machine Learning
- Understanding Naïve Bayes Classifier
- Advantages of Naïve Bayes classifier
- Implementation of Naïve Bayes Classifier

Content

- Introduction to K-Nearest Neighbor
- Distance Measures
- How KNN works?
- How to find k?
- Why KNN?
- Implementation of KNN in Python

Content

What is Naïve Bayes?

Naïve Bayes and Machine Learning

Understanding Naïve Bayes Classifier

Advantages of Naïve Bayes classifier

Implementation of Naïve Bayes Classifier

What is Naïve Bayes?

Let's start with a basic introduction to the Bayes theorem, named after Thomas Bayes from the 1700s.

The Naive Bayes classifier works on the principle of conditional probability, as given by the Bayes theorem.



What is Naïve Bayes?

Let's go through some of the simple concepts of probability that we will use.

Consider the following example of tossing two coins.

If we toss two coins and look at all the different possibilities, we have the sample space as:

{HH, HT, TH, TT}



What is Naïve Bayes?

While calculating the math on probability, we usually denote probability as P.

Some of the probabilities in this event would be as follows:

- The probability of getting two heads = 1/4
- The probability of at least one tail = 3/4
- The probability of the second coin being head given the first coin is tail = 1/2
- The probability of getting two heads given the first coin is a head = 1/2



What is Naïve Bayes?

The Bayes theorem gives us the conditional probability of event A, given that event B has occurred.

In this case, the first coin toss will be B and the second coin toss A.

This could be confusing because we've reversed the order of them and go from B to A instead of A to B.

According to Bayes theorem:

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

where:

$P(A \mid B)$ = Conditional Probability of A given B

$P(B \mid A)$ = Conditional Probability of B given A

$P(A)$ = Probability of event A

$P(B)$ = Probability of event B

What is Naïve Bayes?

Let's apply Bayes theorem to our coin example.

Here, we have two coins, and the first two probabilities of getting two heads and at least one tail are computed directly from the sample space.

Now in this sample space, let A be the event that the second coin is head, and B be the event that the first coin is tails.

Again, we reversed it because we want to know what the second event is going to be.



What is Naïve Bayes?

We're going to focus on A, and we write that out as a probability of A given B:

$$\text{Probability} = P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

$$= \frac{P(\text{First coin being tail given the second coin is head}) \cdot P(\text{Second coin being haed})}{P(\text{First coin being tail})}$$

$$= \frac{0.5 \cdot 0.5}{0.5} = 0.5$$

Bayes theorem calculates the conditional probability of the occurrence of an event based on prior knowledge of conditions that might be related to the event.

Like with any of our other machine learning tools, it's important to understand where the Naive Bayes fits in the hierarchy.

Content

~~What is Naïve Bayes?~~

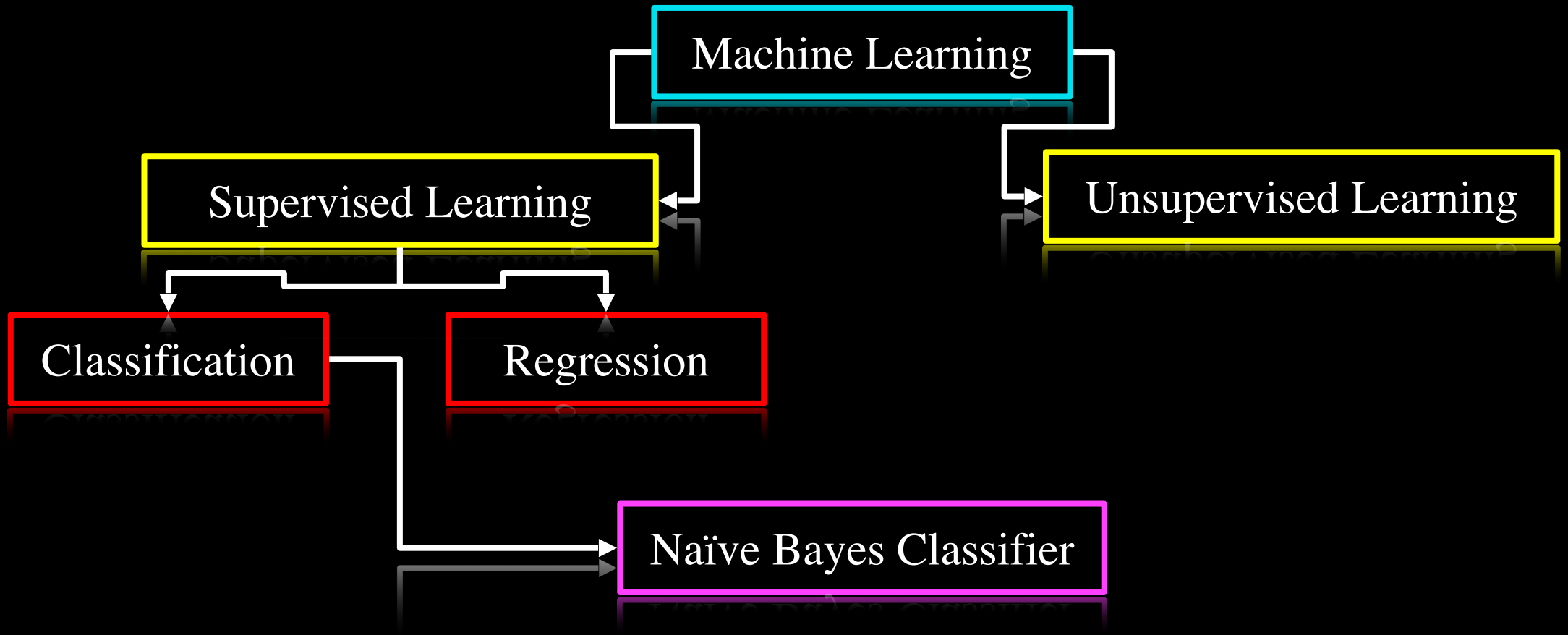
Naïve Bayes and Machine Learning

Understanding Naïve Bayes Classifier

Advantages of Naïve Bayes classifier

Implementation of Naïve Bayes Classifier

Naïve Bayes and Machine Learning



Content

~~What is Naïve Bayes?~~

~~Naïve Bayes and Machine Learning~~

Understanding Naïve Bayes Classifier

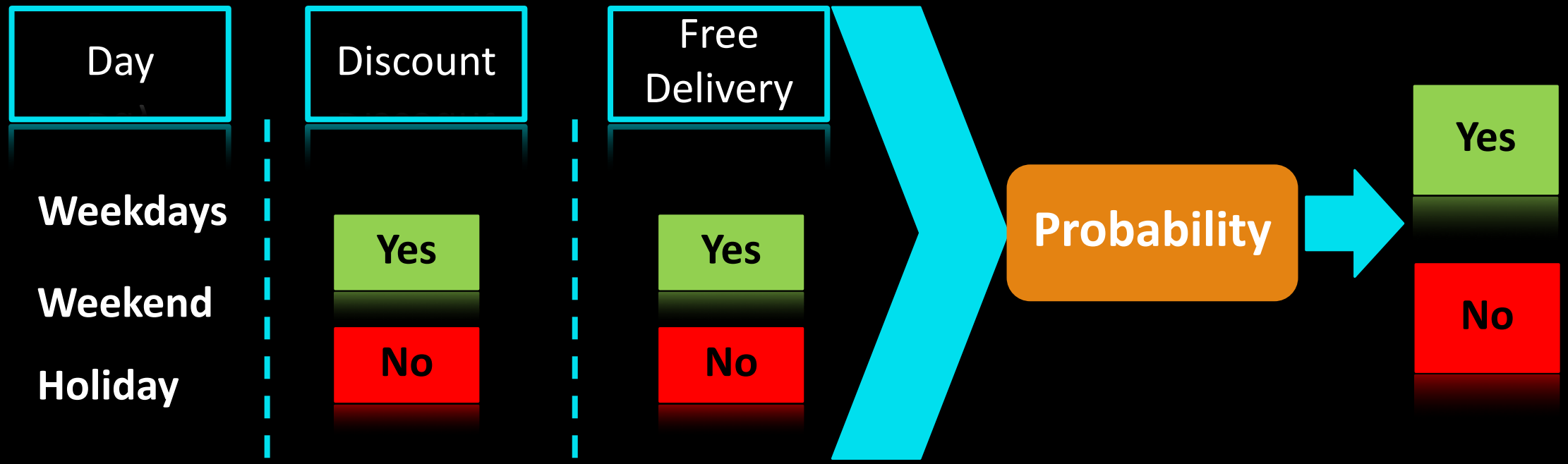
Advantages of Naïve Bayes classifier

Implementation of Naïve Bayes Classifier

Understanding Naïve Bayes Classifier

Shopping Example

Problem statement: To predict whether a person will purchase a product on a specific combination of day, discount, and free delivery using a Naive Bayes classifier.



Understanding Naïve Bayes Classifier

Shopping Example

Under the day, look for variables, like weekday, weekend, and holiday.

For any given day, check if there are a discount and free delivery.

Based on this information, we can predict if a customer would buy the product or not.

See a small sample data set, as shown in Figure:

B18

	A	B	C	D
1	Day	Discount	Free Delivery	Purchase
2	Weekday	Yes	Yes	Yes
3	Weekday	Yes	Yes	Yes
4	Weekday	No	No	No
5	Holiday	Yes	Yes	Yes
6	Weekend	Yes	Yes	Yes
7	Holiday	No	Yes	No
8	Weekend	Yes	No	No
9	Weekday	Yes	Yes	Yes
10	Weekend	Yes	Yes	No
11	Holiday	Yes	Yes	Yes
12	Holiday	No	Yes	Yes
13	Holiday	No	No	No
14	Weekend	Yes	Yes	Yes
15	Holiday	Yes	Yes	Yes
16	Weekday	No	No	Yes
17				

Understanding Naïve Bayes Classifier

Shopping Example

Based on the dataset containing the three input types—day, discount, and free delivery— the frequency table for each attribute is populated.

Frequency Table		Buy	
		Yes	No
Discount	Yes	8	2
	No	2	3

Frequency Table		Buy	
		Yes	No
Day	Weekday	4	1
	Weekend	2	2
	Holiday	4	2

Frequency Table		Buy	
		Yes	No
Free Delivery	Yes	9	2
	No	1	3

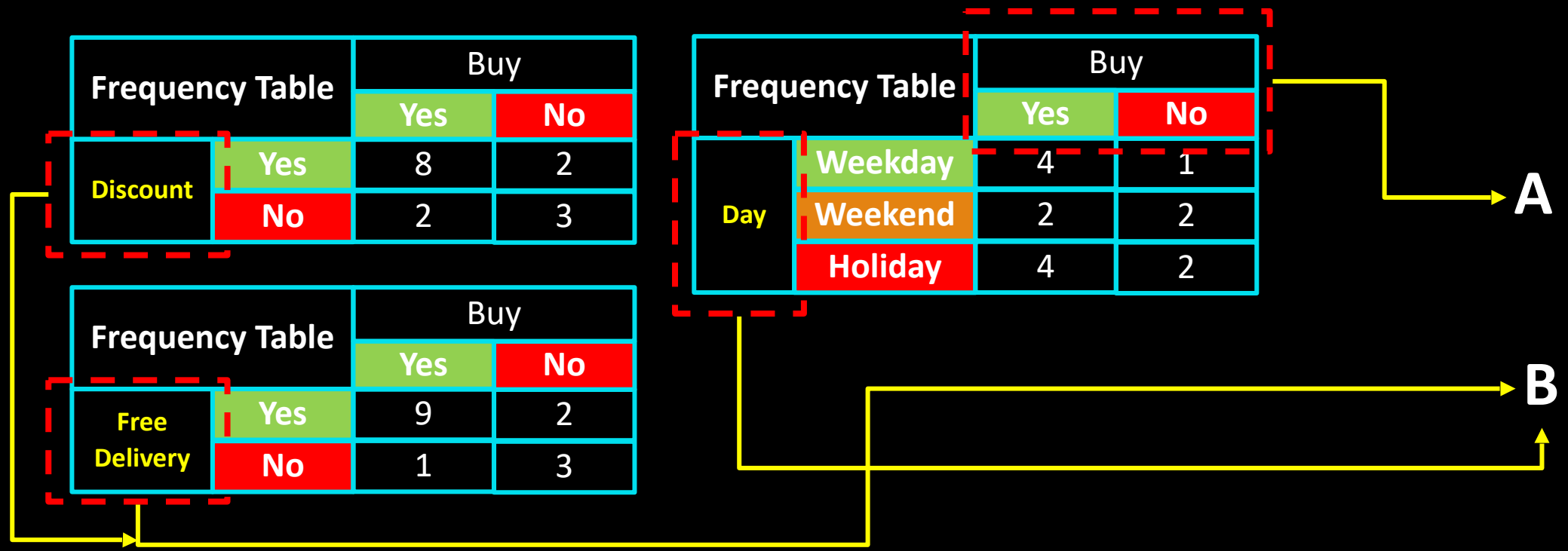
B18

	A	B	C	D
1	Day	Discount	Free Delivery	Purchase
2	Weekday	Yes	Yes	Yes
3	Weekday	Yes	Yes	Yes
4	Weekday	No	No	No
5	Holiday	Yes	Yes	Yes
6	Weekend	Yes	Yes	Yes
7	Holiday	No	Yes	No
8	Weekend	Yes	No	No
9	Weekday	Yes	Yes	Yes
10	Weekend	Yes	Yes	No
11	Holiday	Yes	Yes	Yes
12	Holiday	No	Yes	Yes
13	Holiday	No	No	No
14	Weekend	Yes	Yes	Yes
15	Holiday	Yes	Yes	Yes
16	Weekday	No	No	Yes
17				

Understanding Naïve Bayes Classifier

Shopping Example

For Bayes theorem, let the event ‘buy’ be A and the independent variables (discount, free delivery, day) be B.



Understanding Naïve Bayes Classifier

Shopping Example

Let us calculate the likelihood for one of the "day" variables, which includes weekday, weekend, and holiday variables.

Frequency Table		Buy		
		Yes	No	
Day	Weekday	4	1	5
	Weekend	2	2	4
	Holiday	4	2	6
		10	5	15

Likelihood Table		Buy		
		Yes	No	
Day	Weekday	4/10	1/5	5/15
	Weekend	2/10	2/5	4/15
	Holiday	4/10	2/5	6/15
		10/15	5/15	

Understanding Naïve Bayes Classifier

Shopping Example

We get a total of : 5 weekdays, 4 weekends and 6 holidays

There are 4 out of 10 purchases on weekdays and holidays while 2 out of 10 purchase on weekends.

Based on the above likelihood table, let's calculate some conditional probabilities:

$P(B) = P(Weekday) = 5/15 = 0.33$

$P(A) = P(No\ Buy) = 5/15 = 0.33$

$P(B \mid A) = P(Weekday \mid No\ Buy) = 1/5 = 0.20$

P(A | B) = P(No Buy | Weekday)

= P(Weekday| No Buy) * P(No Buy) / P(Weekday)

= (0.33 * 0.2) / 0.33= 0.20

Likelihood Table		Buy		
		Yes	No	
Day	Weekday	4/10	1/5	5/15
	Weekend	2/10	2/5	4/15
	Holiday	4/10	2/5	6/15
		10/15	5/15	

Understanding Naïve Bayes Classifier

Shopping Example

The probability of purchasing on the weekday = 5/15 or 0.33

It means out of the 30 people who came into the store throughout the weekend, weekday, and holiday, 10 of those purchases were made on weekdays.

The probability of not making a purchase = 5/15 or 0.33. There's a 33 percent chance that they're not going to make a purchase, no matter what day of the week it is.

Likelihood Table		Buy		
		Yes	No	
Day	Weekday	4/10	1/5	5/15
	Weekend	2/10	2/5	4/15
	Holiday	4/10	2/5	6/15
		10/15	5/15	

Understanding Naïve Bayes Classifier

Shopping Example

Finally, we look at the probability of B (i.e., weekdays) when no purchase occurs.

The probability of the weekday without a purchase 1/5= 0.20 or 20 percent.

As the probability of (No | Weekday) is less than 0.5, the customer will most likely buy the product on a weekday. Next, let's see how the table and conditional probabilities work in the Naive Bayes Classifier.

Likelihood Table		Buy		
		Yes	No	
Day	Weekday	4/10	1/5	5/15
	Weekend	2/10	2/5	4/15
	Holiday	4/10	2/5	6/15
		10/15	5/15	

Understanding Naïve Bayes Classifier

Shopping Example

See the likelihood tables for the three variables below:

Frequency Table		Buy	
		Yes	No
Day	Weekday	4	1
	Weekend	2	2
	Holiday	4	2

Frequency Table		Buy	
		Yes	No
Discount	Yes	8	2
	No	2	3

Likelihood Table		Buy		
		Yes	No	
Discount	Yes	8/10	2/5	10/15
	No	2/10	3/5	5/15
		10/15	5/15	

Likelihood Table		Buy		
		Yes	No	
Day	Weekday	4/10	1/5	5/15
	Weekend	2/10	2/5	4/15
	Holiday	4/10	2/5	6/15
		10/15	5/15	

Frequency Table		Buy	
		Yes	No
Free Delivery	Yes	9	2
	No	1	3

Likelihood Table		Buy		
		Yes	No	
Free Delivery	Yes	9/10	2/5	11/15
	No	1/10	3/5	4/15
		10/15	5/15	

Understanding Naïve Bayes Classifier

Shopping Example

The likelihood tables can be used to calculate whether a customer will purchase a product on a specific combination of the day when there is a discount and whether there is free delivery. Consider a combination of the following factors where B equals:

Day = Holiday

Discount = Yes

Free Delivery = Yes

Let's find the probability of them not purchasing based on the conditions above.

A = No Purchase

Understanding Naïve Bayes Classifier

Shopping Example

Applying Bayes Theorem, we get P(A | B) as shown:

$$P(A \mid B) = P(No\ Buy \mid Discount = Yes,\ Free\ Delivery = Yes,\ Day = Holiday) =$$

$$= \frac{P(Disc = Yes \mid No) \bullet P(Free\ Del = Yes \mid No) \bullet P(Day = Holiday \mid No) \bullet P(No\ Buy)}{P(Disc = Yes) \bullet P(Free\ Del = Yes) \bullet P(Day = Holiday)} =$$

$$= \frac{(\frac{2}{5}) \bullet (\frac{2}{5}) \bullet (\frac{2}{5}) \bullet (\frac{5}{15})}{(\frac{10}{15}) \bullet (\frac{11}{15}) \bullet (\frac{6}{15})} = \frac{0.0213}{0.1956} = 0.109$$

Understanding Naïve Bayes Classifier

Shopping Example

Similarly, let us find the probability of them purchasing a product under the conditions above.

Here, A = Buy. Applying Bayes Theorem, we get P(A | B) as shown:

$$P(A \mid B) = P(Yes\ Buy \mid Discount = Yes,\ Free\ Delivery = Yes,\ Day = Holiday) =$$

$$= \frac{P(Disc = Yes \mid Yes) \bullet P(Free\ Del = Yes \mid Yes) \bullet P(Day = Holiday \mid Yes) \bullet P(Yes\ Buy)}{P(Disc = Yes) \bullet P(Free\ Del = Yes) \bullet P(Day = Holiday)} =$$

$$= \frac{(\frac{8}{10}) \bullet (\frac{9}{10}) \bullet (\frac{4}{10}) \bullet (\frac{10}{15})}{(\frac{10}{15}) \bullet (\frac{11}{15}) \bullet (\frac{6}{15})} = \frac{0.192}{0.1956} = 0.982$$

Understanding Naïve Bayes Classifier

Shopping Example

From the two calculations above, we find that:

Probability of purchase = 0.982, Probability of no purchase = 0.109

Finally, we have a conditional probability of purchase on this day.

Next, normalize these probabilities to get the likelihood of the events:

Sum of probabilities = 0.982 + 0.109 = 1.09

Likelihood of purchase = 0.982 / 1.09 = 90 percent

Likelihood of no purchase = 0.109 / 1.09 = 10 percent

Result: As 90 percent is greater than 10 percent, we can conclude that an average customer will buy on holiday with a discount and free delivery.

Content

~~What is Naïve Bayes?~~

~~Naïve Bayes and Machine Learning~~

~~Understanding Naïve Bayes Classifier~~

Advantages of Naïve Bayes classifier

Implementation of Naïve Bayes Classifier

Advantages of Naïve Bayes classifier

The following are some of the benefits of the Naive Bayes classifier:

- It is simple and easy to implement
- It doesn't require as much training data
- It handles both continuous and discrete data
- It is highly scalable with the number of predictors and data points
- It is fast and can be used to make real-time predictions
- It is not sensitive to irrelevant features

Disadvantages of Naïve Bayes classifier

- **Independence assumption**: Assumes features are independent, which rarely matches real-world data.
- **Zero-frequency problem**: Assigns zero probability to unseen categorical test values, needing Laplace smoothing.
- **Poor probability calibration**: Output probabilities can be unreliable (use for ranking/classifying, not true confidence scores).
- **Imbalanced data bias**: Skews toward majority classes on heavily skewed datasets.

Content

~~What is Naïve Bayes?~~

~~Naïve Bayes and Machine Learning~~

~~Understanding Naïve Bayes Classifier~~

~~Advantages of Naïve Bayes classifier~~

Implementation of Naïve Bayes Classifier

Implementation of Naïve Bayes Classifier

```python
# Function to calculate the likelihood P(feature|class)
def likelihood(df, feature_name, feature_value, target_name, target_value):
    filtered_df = df[df[target_name] == target_value]
    count_feature_given_class = len(filtered_df[filtered_df[feature_name] == feature_value])
    count_class = len(filtered_df)
    return count_feature_given_class / count_class if count_class != 0 else 0

# Function to calculate the prior probability P(class)
def prior(df, target_name, target_value):
    count_class = len(df[df[target_name] == target_value])
    return count_class / len(df)

# Function to calculate the Naive Bayes posterior P(class|features)
def naive_bayes(df, features, target_name, target_value):
    # Start with the prior probability
    prob = prior(df, target_name, target_value)

    # Multiply by the likelihood for each feature
    for feature_name, feature_value in features.items():
        prob *= likelihood(df, feature_name, feature_value, target_name, target_value)

    return prob
```

[7] ✓ 0.0s

Implementation of Naïve Bayes Classifier

```
# Define the conditions for prediction (Day=Holiday, Discount=Yes, Free Delivery=Yes)
conditions = {
    'Day': 'Holiday',
    'Discount': 'Yes',
    'Free Delivery': 'Yes'
}

# Calculate the probabilities for Purchase = Yes and Purchase = No
prob_yes = naive_bayes(dataset, conditions, 'Purchase', 'Yes')
prob_no = naive_bayes(dataset, conditions, 'Purchase', 'No')

# Normalize the probabilities to make them sum to 1
total_prob = prob_yes + prob_no
prob_yes_normalized = prob_yes / total_prob
prob_no_normalized = prob_no / total_prob

# Print the probabilities
print(f"Probability of Purchase (Yes): {prob_yes_normalized * 100:.2f}%")
print(f"Probability of No Purchase: {prob_no_normalized * 100:.2f}%")

# Predict the outcome based on the higher probability
prediction = 'Yes' if prob_yes_normalized > prob_no_normalized else 'No'
print(f"Prediction: The customer will {'purchase' if prediction == 'Yes' else 'not purchase'} the product.")
```

[8] ✓ 0.0s

Content

- Introduction to K-Nearest Neighbor
- Distance Measures
- How KNN works?
- How to find k?
- Why KNN?
- Implementation of KNN in Python

Introduction to K-Nearest Neighbor

- The KNN algorithm predicts responses for new data (testing data) based upon its similarity with other known data (training) samples. It assumes that data with similar traits sit together and uses distance measures at its core.

- KNN belongs to the class of non-parametric models as it doesn't learn parameters from the training dataset to come up with a discriminative function to predict new unseen data. It operates by memorizing the training dataset.

Content

~~Introduction to K-Nearest Neighbor~~

Distance Measures

How KNN works?

How to find k?

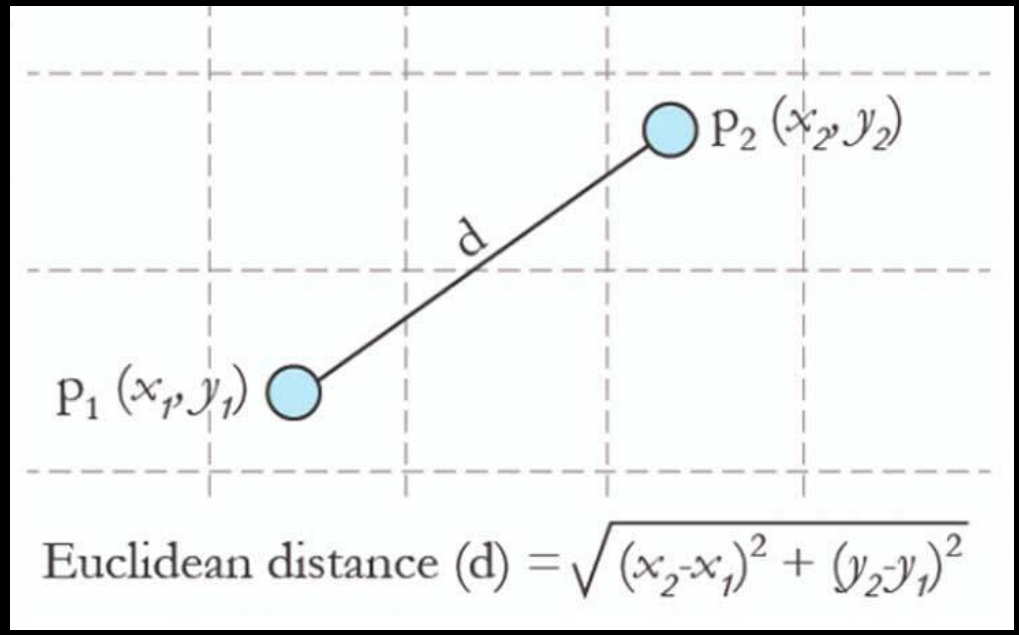
Why KNN?

Implementation of KNN in Python

Distance Measures

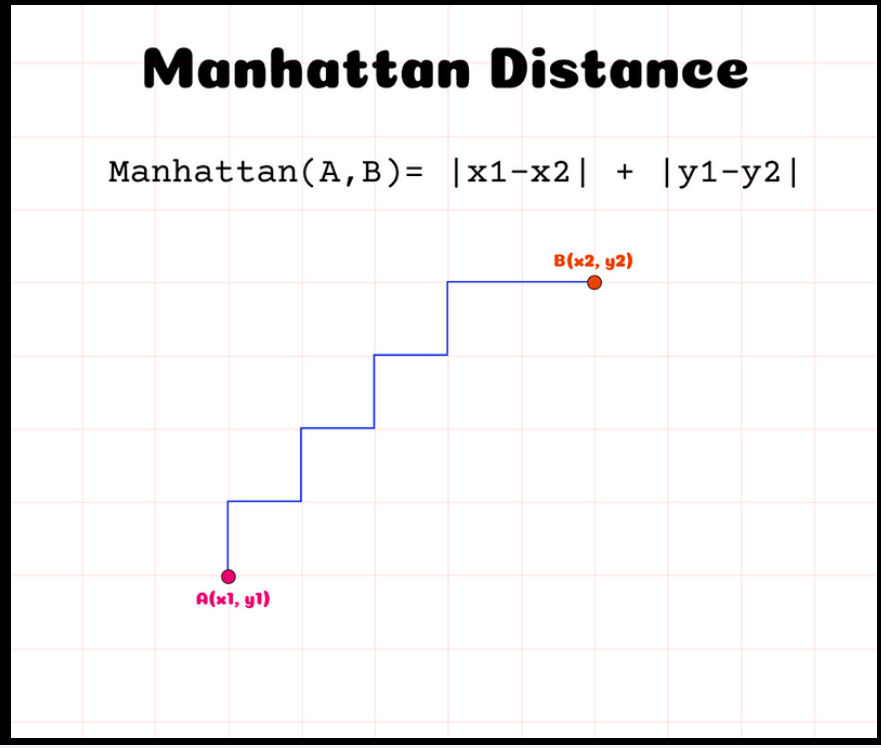
- Euclidean is probably the most intuitive one and represents the shortest distance between two points. It's calculated using the well-known Pythagorean theorem.

- Conceptually, it should be used whenever we are comparing observations with continuous features, like height, weight, or salaries. This distance measure is often the "default" distance used in algorithms like KNN.



Distance Measures

- Manhattan is used to estimate the distance to get from one data point to another if a grid-like path is taken. Unlike Euclidean distance, the Manhattan distance calculates the sum of the absolute values of the difference of the coordinates of two points. This way, instead of estimating a straight line between two points, we "walk" through available paths.
- The Manhattan distance is useful when our observations are distributed along a grid (when the features of our observations are entire integers with no decimal parts).



$$\text{Manhattan Distance} = d(x,y) = \left(\sum_{i=1}^{m} |x_i - y_i|\right)$$

Distance Measures

- Minkowski distance: is a generalization of Euclidean and Manhattan distance metrics, which enables the creation of other distance metrics. It is calculated in a normed vector space.

- In the Minkowski distance, p is the parameter that defines the type of distance used in the calculation. If p=1, then the Manhattan distance is used. If p=2, then the Euclidean distance is used.

$$\text{Minkowski Distance} = \left(\sum_{i=1}^{n} |x_i - y_i|\right)^{1/p}$$

Content

~~Introduction to K-Nearest Neighbor~~

~~Distance Measures~~

How KNN works?

How to find k?

Why KNN?

Implementation of KNN in Python

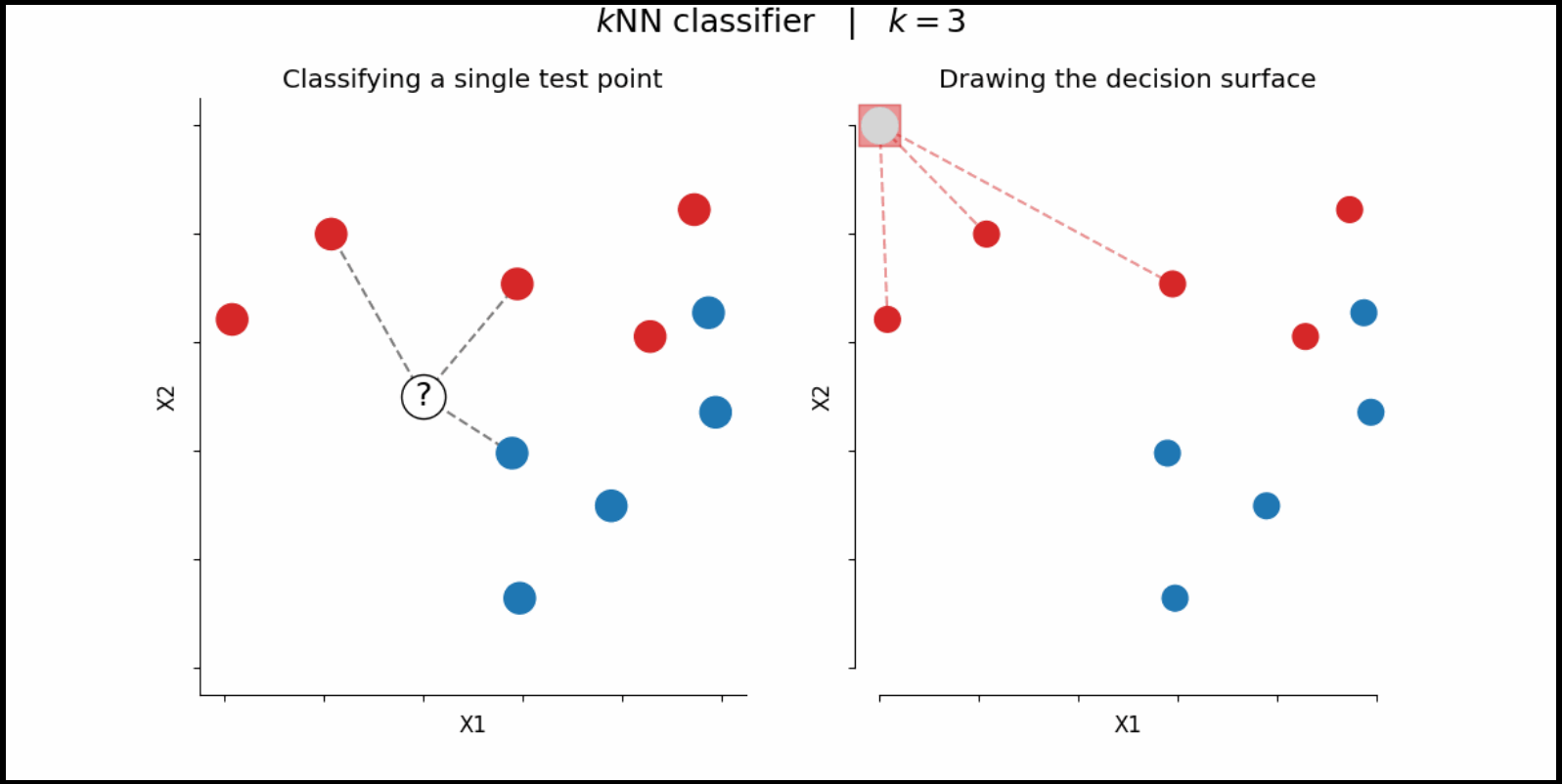
How KNN works?

- After the user defines a distance function, like the ones we mentioned earlier, KNN calculates the distance between data points in order to find the closest data points from our training data for any new data point. The existing data points closest to the new data point using the defined distance will become the "k-neighbors".

- For a classification task, KNN will use the most frequent of all values from the k-neighbors to predict the new data label.

- For a regression task, the algorithm will use the average of all values from the k-neighbors to predict the new data value.

How KNN works?



Source: https://commons.wikimedia.org/wiki/File:KNN_decision_surface_animation.gif

How KNN works?

- All KNN does is store the complete dataset, and without doing any calculation or modeling on top of it, measure the distance between the new data point and its closest data points.

- For this reason, and since there's not really a learning process happening, KNN is called a "lazy" algorithm.

How KNN works?

Sepal Length	Sepal Width	Species
5.3	3.7	Setosa
5.1	3.8	Setosa
7.2	3.0	Virginica
5.4	3.4	Setosa
5.1	3.3	Setosa
5.4	3.9	Setosa
7.4	2.8	Virginica
6.1	2.8	Vercicolor
7.3	2.9	Virginica
6.0	2.7	Vercicolor
5.8	2.8	Virginica
6.3	2.3	Vercicolor

Sepal Length	Sepal Width	Species
5.2	3.1	?

Step 1: Find Distence

$$D(Sepal\ Length,\ Sepal\ Width) = \sqrt{(x-a)^2 + (y-b)^2}$$

$$D(Sepal\ Length,\ Sepal\ Width) = \sqrt{(5.2-5.3)^2 + (3.1-3.7)^2}$$

$$D(Sepal\ Length,\ Sepal\ Width) = 0.608$$

Sepal Length	Sepal Width	Species	Distance
5.3	3.7	Setosa	0.608

How KNN works?

Sepal Length	Sepal Width	Species	Distance
5.3	3.7	Setosa	0.608
5.1	3.8	Setosa	0.707
7.2	3.0	Virginica	2.002
5.4	3.4	Setosa	0.36
5.1	3.3	Setosa	0.22
5.4	3.9	Setosa	0.82
7.4	2.8	Virginica	2.22
6.1	2.8	Vercicolor	0.94
7.3	2.9	Virginica	2.1
6.0	2.7	Vercicolor	0.89
5.8	2.8	Virginica	0.67
6.3	2.3	Vercicolor	1.36

Step 2: Find Rank

How KNN works?

Sepal Length	Sepal Width	Species	Distance	Rank
5.3	3.7	Setosa	0.608	3
5.1	3.8	Setosa	0.707	5
7.2	3.0	Virginica	2.002	10
5.4	3.4	Setosa	0.36	2
5.1	3.3	Setosa	0.22	1
5.4	3.9	Setosa	0.82	6
7.4	2.8	Virginica	2.22	12
6.1	2.8	Vercicolor	0.94	8
7.3	2.9	Virginica	2.1	11
6.0	2.7	Vercicolor	0.89	7
5.8	2.8	Virginica	0.67	4
6.3	2.3	Vercicolor	1.36	9

Step 3: Find Nearest Neighbor

If K=3

Sepal Length	Sepal Width	Species	Distance	Rank
5.1	3.3	Setosa	0.22	1
5.4	3.4	Setosa	0.36	2
5.3	3.7	Setosa	0.608	3

Sepal Length	Sepal Width	Species
5.2	3.1	?

Sepal Length	Sepal Width	Species
5.2	3.1	Setosa

Content

~~Introduction to K-Nearest Neighbor~~

~~Distance Measures~~

~~How KNN works?~~

How to find k?

Why KNN?

Implementation of KNN in Python

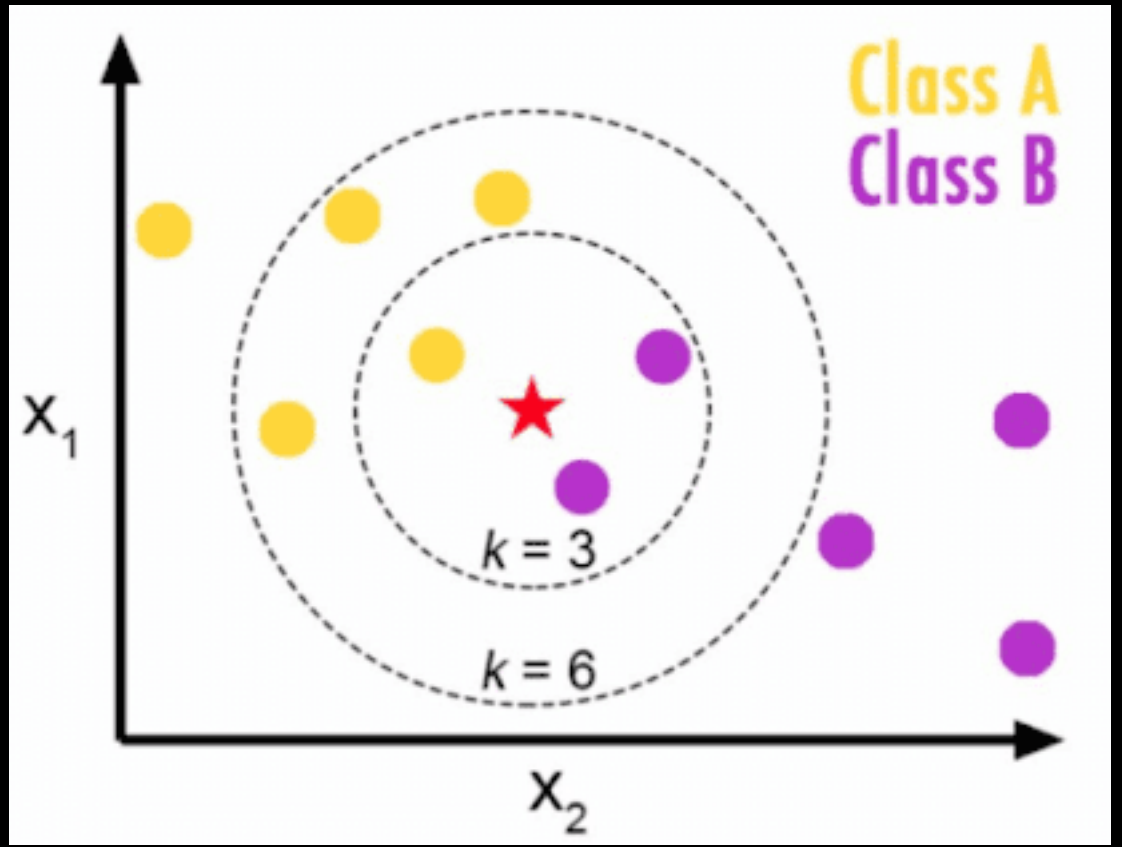
How to find k?

- With KNN, in order to make a classification/regression task, you need to define a number of neighbors, and that number is given by the parameter “k”. In other words, “k” determines the number of neighbors the algorithm looks at when assigning a value to any new observation.

- This number can go from 1 (in which case the algorithm only looks at the closest neighbor for each prediction) to the total number of data points of the dataset (in which case the algorithm would predict the majority class of the complete dataset).

How to find k?

- In the above example, if k=3 then the new data point will be classified as B.
- But if k=6, then it will be classified as A because the majority of points are from class A.
- Source: ShareTechnote



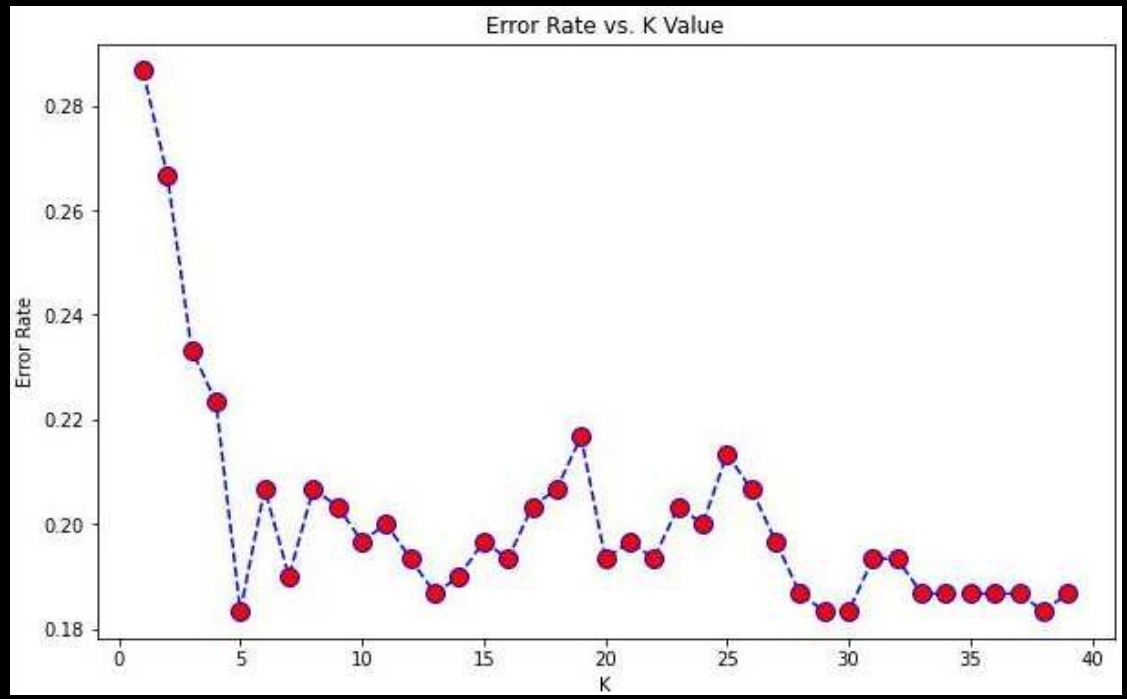
How to find k?

- So how can you know the optimum value of “k”? We can decide based on the error calculation of a training and testing set. Separating the data into training and test sets allows for an objective model evaluation.

- One popular approach is testing different numbers of “k” and measuring the resulting error, choosing the “k” value at which an increase will cause a very small decrease in the error sum, while a decrease will sharply increase the error sum. This point that defines the optimal number is known as the “elbow point”.

How to find k?

- The “k” value best suited for a dataset is selected by using the error rate (this is, the difference between the real and predicted values). After a certain point, the error rate becomes almost constant.
- We can see that after “k” > 5, the error rate stabilizes and becomes almost constant. For this reason, “k” = 5 seems to be the optimal value.



Content

~~Introduction to K-Nearest Neighbor~~

~~Distance Measures~~

~~How KNN works?~~

~~How to find k?~~

Why KNN?

Implementation of KNN in Python

Why KNN?

Advantages of the kNN algorithm

The kNN algorithm is often described as the "simplest" supervised learning algorithm, which leads to its several advantages:

- ✓ Simple: kNN is easy to implement because of how simple and accurate it is. As such, it is often one of the first classifiers that a data scientist will learn.
- ✓ Adaptable: As soon as new training samples are added to its dataset, the kNN algorithm adjusts its predictions to include the new training data.
- ✓ Easily programmable: kNN requires only a few hyper-parameters — a k value and a distance metric. This makes it a fairly uncomplicated algorithm.

Why KNN?

Challenges and limitations of kNN

While the kNN algorithm is simple, it also has a set of challenges and limitations, due in part to its simplicity:

- ✓ Difficult to scale: Because kNN takes up a lot of memory and data storage, it brings up the expenses associated with storage. This reliance on memory also means that the algorithm is computationally intensive, which is in turn resource-intensive.

Why KNN?

Challenges and limitations of kNN

- ✓ Curse of dimensionality: This refers to a phenomenon that occurs in computer science, wherein a fixed set of training examples is challenged by an increasing number of dimensions and the inherent increase of feature values in these dimensions. In other words, the model's training data can't keep up with the evolving dimensionality of the hyperspace. This means that predictions become less accurate because the distance between the query point and similar points grows wider — on other dimensions.
- ✓ Overfitting: The value of k, as shown earlier, will impact the algorithm's behavior. This can happen especially when the value of k is too low. Lower values of k can overfit the data, while higher values of k will 'smooth' the prediction values because the algorithm averages values over a greater area.

Content

~~Introduction to K-Nearest Neighbor~~

~~Distance Measures~~

~~How KNN works?~~

~~How to find k?~~

~~Why KNN?~~

Implementation of KNN in Python

Implementation of KNN in Python

```python
# K-Nearest Neighbors implementation
class KNN:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X_train, y_train):
        self.X_train = X_train
        self.y_train = y_train

    def euclidean_distance(self, x1, x2):
        return np.sqrt(np.sum((x1 - x2) ** 2))

    def predict(self, X_test):
        y_pred = [self._predict(x) for x in X_test]
        return np.array(y_pred)

    def _predict(self, x):
        # Compute distances between x and all examples in the training set
        distances = [self.euclidean_distance(x, x_train) for x_train in self.X_train]
        # Sort by distance and return the indices of the first k neighbors
        k_indices = np.argsort(distances)[:self.k]
        # Extract the labels of the k nearest neighbor training samples
        k_nearest_labels = [self.y_train[i] for i in k_indices]
        # Return the most common class label
        most_common = np.bincount(k_nearest_labels).argmax()
        return most_common
```

```python
# Instantiate KNN classifier and make predictions
knn = KNN(k=3)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

print("Accuracy:", accuracy)
print("Confusion Matrix:\n", conf_matrix)
```




THANK YOU